ROOTCAUSE SERVER

Documentación del sistema — RootCause Server

Documento: README.md **Versión analizada:** 0.2.0 **Commit:** 2efedb2 **Fecha del análisis:** 2026-08-26

CONTENIDO

El sistema, en una frase
Propósito de esta documentación
Tabla de contenidos
  Recursos
Documentación previa del repositorio
Datos del análisis
Convenciones
Pendiente de validar

Portada e índice navegable de la documentación de análisis, arquitectura, código, datos, seguridad y operación del repositorio `rootcause-server`.

El sistema, en una frase

RootCause Server es un plano de control escrito en Rust que vigila un conjunto de servidores y la red que los rodea: recibe telemetría de un agente de solo lectura instalado en cada equipo, aplica 18 reglas de detección deterministas, y presenta cada hallazgo con su evidencia, su causa y un *runbook* que nunca ejecuta por su cuenta.

Propósito de esta documentación

Convertir la lectura completa del repositorio en material que sirva a cinco públicos distintos sin obligar a ninguno a leer el código fuente:

Público	Empieza por
Persona ajena al sistema	01 · Descripción general y 17 · Resumen ejecutivo
Desarrollador que se incorpora	18 · Guía para un nuevo desarrollador
Desarrollador experimentado	05 · Referencia técnica y 06 · Explicación profunda del código
Auditor	11 · Seguridad, 15 · Riesgos y deuda técnica, 19 · Matriz de trazabilidad
Agente de IA	04 · Mapa del código y 19 · Matriz de trazabilidad

Tabla de contenidos

#	Documento	Contenido	Estado
—	README	Portada, índice y convenciones	Completo
01	Descripción general del sistema	Qué es, qué resuelve, para quién, con una sección no técnica	Completo

#	Documento	Contenido	Estado
02	Instalación y ejecución	Requisitos, compilación, arranque, pruebas, errores frecuentes	Completo
03	Arquitectura	Estilo, capas, patrones y siete diagramas Mermaid	Completo
04	Mapa completo del código	Inventario jerárquico de directorios, módulos y símbolos	Completo
05	Referencia técnica	Catálogo de tipos, funciones, variables de entorno, endpoints y errores	Completo
06	Explicación profunda del código	Flujo interno, condicionales y casos límite módulo por módulo	Completo
07	Base de datos	Motor, esquema, diccionario de datos, migraciones y diagrama E-R	Completo
08	Flujo de datos	Origen, validación, transformación, almacenamiento y consumo	Completo
09	APIs e integraciones	Contrato HTTP, códigos de estado, Prometheus y exportación	Completo
10	Configuración	Variables, valores por omisión y consecuencias de equivocarse	Completo
11	Seguridad	Autenticación, perímetro, cabeceras, secretos y controles ausentes	Completo
12	Pruebas y calidad	Tipos de prueba, cobertura, CI y pruebas faltantes priorizadas	Completo
13	Despliegue y operación	Entornos, contenedor, servicios del sistema, CI/CD y respaldo	Completo
14	Solución de problemas	Síntoma, causa, diagnóstico y solución	Completo
15	Riesgos y deuda técnica	Hallazgos clasificados por severidad, impacto y evidencia	Completo
16	Glosario	Términos técnicos y de dominio en lenguaje comprensible	Completo
17	Resumen ejecutivo	Presentación para decisión, sin detalle innecesario	Completo
18	Guía para un nuevo desarrollador	Itinerario de incorporación y primeras tareas	Completo
19	Matriz de trazabilidad	De la funcionalidad a la tabla y a la prueba	Completo

Recursos

- `assets/` — material gráfico de apoyo. Los diagramas viven embebidos en Mermaid dentro de los documentos, así que hoy esta carpeta contiene solo la nota que lo explica.
- `pdf/` — los 20 documentos de esta carpeta en PDF, generados desde el Markdown —que es la única fuente— con `scripts/docs_pdf.py`:

``` bash python3 scripts/docs_pdf.py ```

El script convierte el Markdown a HTML con la biblioteca estándar de Python, dibuja los diagramas Mermaid con `mmdc` y manda imprimir con Chrome, Chromium o Edge en modo headless. Si falta `mmdc` o el navegador, avisa y entrega lo que sí puede producir en lugar de fallar en silencio.

Documentación previa del repositorio

Esta carpeta **no reemplaza** la documentación que el proyecto ya mantenía en `docs/`: la referencia, la enlaza y la amplía. Los documentos originales siguen siendo la fuente autoritativa de su tema.

Documento existente	Tema
`docs/ARCHITECTURE.md`	Arquitectura, tal como la mantiene el equipo
`docs/API.md`	Contrato del API
`docs/CAPABILITIES.md`	Matriz de capacidades y límites declarados
`docs/DETECCION_AMENAZAS.md`	Catálogo de amenazas detectadas
```

Documento existente	Tema
`docs/HARDENING.md`	Endurecimiento del despliegue
`docs/INSTALLATION.md`	Instalación multiplataforma
`docs/MANUAL_USUARIO.md`	Manual de operación
`docs/THREAT_MODEL.md`	Modelo de amenazas
`docs/SECURITY_REQUIREMENTS.md`	Requisitos de seguridad
`docs/CROSS_PLATFORM.md`	Diferencias entre Windows, Linux y macOS
`docs/POLITICA_DE_PRIVACIDAD_LOCAL.md`	Qué datos se procesan y dónde quedan
`docs/ROADMAP.md`	Hoja de ruta
`docs/MASTER_PROMPT.md`	Prompt maestro del proyecto
`docs/adr/`	Decisiones de arquitectura registradas
`docs/requirements/`	REQ-SEC-001, 002 y 003

Datos del análisis

Dato	Valor
Fecha del análisis	2026-08-26
Versión analizada	`0.2.0` (`Cargo.toml`, `[workspace.package]`)
Commit analizado	`2efedb2180dcecac6dc66e8e48a2cce90ebc6c3d`
Rama	`main`
Estado del árbol de trabajo	Limpio al iniciar el análisis
Lenguaje principal	Rust `1.95.0` (edición 2024)
Lenguajes auxiliares	JavaScript (consola), Python (guardianes y utilidades), SQL, HTML, CSS
Archivos de código analizados	33 archivos Rust, 4 de consola, 4 scripts Python, 2 migraciones SQL
Reglas de detección publicadas	18
Anotaciones de prueba en el código	250 (`#[test]` y `#[tokio::test]`)

Convenciones

Marcadores de certeza. Cuando una afirmación no se pudo comprobar contra el repositorio, aparece marcada:

Marcador	Significado
No identificado	Se buscó y no se encontró nada en el repositorio
No documentado en el repositorio	Existe en el código, pero el repositorio no explica la intención
Requiere validación	Solo puede confirmarlo una persona con acceso al entorno real
Inferencia basada en el código	Conclusión razonada a partir del código, no una afirmación del repositorio

Otras convenciones.

- Los nombres de archivos, funciones, tipos, tablas y columnas se conservan literales, para que cualquier afirmación se pueda verificar con `grep`.
- Las rutas se escriben relativas a la raíz del repositorio.
- Los comandos son ejecutables tal cual; los valores sensibles siempre son ficticios (`reemplaza-por-...`, `203.0.113.10`, `pega-aqui-el-token`).
- Idioma: documentación en español, igual que el resto del proyecto. El código y sus comentarios están en inglés, por decisión del propio repositorio (`AGENTS.md`).
- Los diagramas van siempre acompañados de su explicación en texto: ningún diagrama es la única fuente de una idea.

Pendiente de validar

Lista viva de lo que este análisis **no** pudo confirmar leyendo el repositorio. Cada punto está desarrollado en el documento que lo menciona.

#	Pendiente	Dónde se detalla
1	El directorio `config/` existe en la copia de trabajo pero **no está en el repositorio** (git no versiona directorios vacíos)	04, 15
2	No hay respaldo ni restauración automatizados del archivo SQLite	07, 13
3	La firma y la atestación de procedencia de los artefactos de publicación no se pudieron verificar sin ejecutar el workflow	13
4	El comportamiento de los parsers frente a salidas de `ss` y `netstat` no cubiertas por fixtures	12, 15
5	El instalador de Windows (`packaging/windows/rootcause-server.iss`) no se compiló en este análisis	13

Generado por análisis directo del repositorio el 2026-08-26 sobre el commit `2efedb2`. Toda cifra de este conjunto documental proviene del código, no de la documentación previa.
